

1 Abstract

This report details the initial phase of a data analysis workflow designed to process high-frequency High-Performance Liquid Chromatography (HPLC) signals for 11 distinct experimental runs. The primary objective was to establish a robust pipeline for baseline correction, artifact exclusion, and the inference of chromatographic stages to facilitate subsequent peak detection and quantification. While the data preprocessing successfully handled approximately 1.1 million rows and applied Asymmetric Least Squares (AsLS) baseline correction, a critical failure was identified in the elution window inference algorithm. The dynamic threshold used to identify 'Elution' stages based on the gradient of the mobile phase composition ('Conc.B_%') proved too stringent, resulting in the near-total misclassification of data points as 'Unknown'. Consequently, the downstream steps for peak detection and integration could not be executed as planned. This report outlines the methodology, highlights the specific algorithmic failure in stage inference, and discusses the immediate need for parameter recalibration to salvage the analysis.

2 Introduction

The analysis of HPLC chromatograms requires rigorous preprocessing to distinguish true analyte peaks from baseline noise and solvent artifacts. In this study, the dataset comprised high-resolution UV signals ('UV_1_280_mAU') across 11 runs, characterized by significant missingness (75%) in the 'chromatography_stage' metadata. To address this, the analysis plan mandated an automated inference of process stages, specifically the 'Elution' phase, which is critical for defining the integration windows for peak detection. The workflow was designed to first correct the baseline using Asymmetric Least Squares (AsLS) on pre-injection regions, exclude the solvent front (volume $\leq$ 1.0 ml), and then infer elution windows based on the slope of the organic modifier concentration ('Conc.B_%'). The ultimate goal of this workflow is to calculate the relative proportion of compounds per run by integrating peak areas and normalizing them against the total valid area. However, the validity of the entire downstream analysis depends on the accurate identification of the elution window. If this window is not correctly inferred, peak detection algorithms will either integrate noise or miss significant analyte signals entirely.

3 Methodology

The data analysis followed a three-step plan, with the current report focusing on the execution and results of Step 1: Data Preprocessing, Baseline Correction, and Elution Window Inference.

1. **Data Loading and Baseline Correction**: The raw data ('chromatography_combined.csv') was loaded and grouped by 'run_no'. To handle pre-injection artifacts, negative 'volume_ml' values were retained initially to estimate the baseline. An Asymmetric Least Squares (AsLS) algorithm was applied to the 'UV_1_280_mAU' signal using 'volume_ml' as the x-axis, with parameters set to $\lambda=1e6$ and $p=0.001$. The baseline was estimated using the pre-injection region and subsequently subtracted from the entire signal.

2. **Artifact Exclusion**: Following baseline correction, rows where 'volume_ml' was less than 1.0 ml were filtered out to exclude the solvent front. This step varied by run, removing between approximately 2,200 and 6,500 rows per run.

3. **Elution Window Inference**: For rows where 'chromatography_stage' was missing, an inference logic was applied. The algorithm calculated the rolling slope of 'Conc.B_%' (window size = 5) and normalized it by the 'System_flow_ml_min'. A row was labeled as 'Elution' only if the normalized slope exceeded a dynamic threshold ($\geq 2\%$ per ml, adjusted for flow) and the volume was greater than 5 ml. All other rows were explicitly labeled as 'Unknown'.

4. **Downstream Plan**: The intended subsequent steps (Step 2 and 3) involved filtering data to the 'Elution' stage, detecting peaks using 'scipy.signal.find_peaks' with height and width thresholds, integrating areas at 280 nm and 260 nm for purity validation, and calculating run-level relative proportions. However, these steps were contingent on the success of the inference logic in Step 1.

4 Results and Discussion

The execution of Step 1 successfully processed 11 chromatographic runs, comprising a total dataset of approximately 1.1 million rows. The baseline correction and solvent front exclusion were performed without error, with the number of excluded rows varying logically across runs based on the length of the pre-injection and solvent front regions.

However, the results of the elution window inference revealed a critical methodological failure. Across all 11 runs, only 9 rows were successfully labeled as 'Elution' (2 in run 16, 5 in run 30, and 7 in run 32). The vast majority of the dataset, ranging from approximately 9,700 to 120,000 rows per run, was labeled as 'Unknown'. This outcome indicates that the dynamic threshold for inferring elution based on the normalized slope of 'Conc_B_%' is significantly misaligned with the actual gradient profiles present in the dataset. The threshold of $\geq 2\%$ per ml appears to be too stringent, failing to capture the gradual or specific gradient changes characteristic of the elution phase in these experiments.

This failure has severe implications for the data analysis workflow. Step 2, which relies on filtering data to include only rows labeled 'Elution' for peak detection, cannot proceed effectively. Without a robust identification of the elution window, the peak detection algorithm would either operate on an empty dataset or, if the filter were bypassed, risk integrating noise and wash artifacts as valid peaks. The raw signal quality appears intact, as evidenced by the high row counts and successful baseline subtraction, but the validity of the analysis is currently compromised by the inability to isolate the relevant chromatographic phases.

To resolve this, the slope threshold must be recalibrated, or alternative heuristics (such as absolute concentration thresholds or fixed volume windows based on experimental design) must be incorporated to correctly classify the elution stage. Until this inference logic is corrected, the calculation of peak areas, purity ratios, and run-level relative proportions remains impossible.

5 Conclusion

The initial phase of the data analysis workflow successfully demonstrated the capability to preprocess high-volume HPLC data, apply AsLS baseline correction, and exclude solvent front artifacts. However, the workflow encountered a critical bottleneck in the automated inference of chromatographic stages. The current algorithm for detecting the 'Elution' phase failed to identify valid windows in 11 out of 11 runs, labeling nearly the entire dataset as 'Unknown'. This failure prevents the execution of subsequent peak detection and integration steps, rendering the primary objective of calculating relative compound proportions unachievable in its current state. Immediate action is required to recalibrate the slope-based inference threshold or to implement alternative logic for stage detection. Once the elution windows are correctly identified, the workflow can proceed to peak integration, purity validation, and comparative analysis.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy.sparse import diags, csr_matrix
from scipy.sparse.linalg import spsolve

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_LOG = '/workspace/preprocessing_log.md'
VOLUME_FILTER_THRESHOLD = 1.0
ASLS_LAMBDA = 1e6
ASLS_P = 0.001
```

```

SLOPE_WINDOW = 5
SLOPE_THRESHOLD_PERCENT = 2.0
MIN_VOLUME_FOR_ELUTION = 5.0

def asymmetric_least_squares_baseline(y, lam, p):
    """
    Asymmetric Least Squares Baseline Correction using Sparse Iterative Solver.
    """
    n = len(y)
    if n < 3:
        return y

    # Construct second derivative matrix D (n-2 x n) with [1, -2, 1]
    diagonals = [np.ones(n-2), -2 * np.ones(n), np.ones(n-2)]
    offsets = [0, 1, 2]
    D = diags(diagonals, offsets, shape=(n-2, n), format='csr')

    # Precompute D.T @ D for the penalty term
    DTD = D.T @ D

    W = np.ones(n)
    z = y.copy()

    for _ in range(10):
        W_mat = diags(W, 0, shape=(n, n), format='csr')
        A = W_mat + lam * DTD
        b = W * y
        z = spsolve(A, b)
        W = p * (y > z) + (1 - p) * (y <= z)

    return z

def process_chromatography_data():
    required_cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%',
                    'chromatography_stage', 'Fraction_unique_ID', '
                    System_flow_ml_min']

    df = pd.read_csv(INPUT_FILE, usecols=required_cols)

    results_summary = []
    df = df.sort_values(['run_no', 'volume_ml'])

    os.makedirs(os.path.dirname(OUTPUT_LOG), exist_ok=True)

    for run_id, group in df.groupby('run_no'):
        # Work on a copy for this run to avoid SettingWithCopyWarning
        run_data = group.copy()
        total_rows = len(run_data)

        # 1. Apply AsLS baseline correction on FULL dataset (including negative
        volumes)
        uv_signal = run_data['UV_1_280_mAU'].values
        baseline = asymmetric_least_squares_baseline(uv_signal, ASLS_LAMBDA,
            ASLS_P)
        run_data['UV_1_280_mAU_corrected'] = uv_signal - baseline

        # 2. Filter out rows where volume_ml < 1.0 AFTER baseline correction
        rows_before_filter = len(run_data)
        run_data = run_data[run_data['volume_ml'] >= VOLUME_FILTER_THRESHOLD]

```

```

rows_excluded = rows_before_filter - len(run_data)

# 3. Infer chromatography_stage
flow = run_data['System_flow_ml_min'].values
conc_b = run_data['Conc_B_%'].values

# Calculate rolling slope using polyfit for accuracy on non-linear
# segments
# Note: This uses rolling().apply() which is slower than vectorized shift
# but more accurate
conc_b_series = pd.Series(conc_b)
rolling_slope = conc_b_series.rolling(window=SLOPE_WINDOW).apply(
    lambda x: np.polyfit(range(len(x)), x, 1)[0] if not np.isnan(x).any()
    else np.nan,
    raw=False
)

# Normalize by flow
# If System_flow_ml_min is zero, skip normalization (use raw slope)
normalized_slope = np.where(flow == 0, rolling_slope.values, rolling_slope
    .values / flow)

# Condition: normalized slope > threshold AND volume_ml > 5 ml
is_elution = (normalized_slope > SLOPE_THRESHOLD_PERCENT) & (run_data['
    volume_ml'] > MIN_VOLUME_FOR_ELUTION)

# Handle Fraction_unique_ID continuity:
# The plan requires treating missing Fraction_unique_ID as continuous flow
# segments.
# We ensure this by grouping by run_no (already done) and not breaking the
# rolling window
# based on missing IDs. The rolling window operates on the sorted volume
# sequence.

missing_mask = run_data['chromatography_stage'].isna()

inferred_elution = is_elution & missing_mask
inferred_unknown = (~is_elution) & missing_mask

run_data.loc[inferred_elution, 'chromatography_stage'] = 'Elution'
run_data.loc[inferred_unknown, 'chromatography_stage'] = 'Unknown'

num_elution = run_data['chromatography_stage'].eq('Elution').sum()
num_unknown = run_data['chromatography_stage'].eq('Unknown').sum()

results_summary.append({
    'run_no': run_id,
    'total_rows_processed': total_rows,
    'rows_excluded_post_baseline': rows_excluded,
    'rows_inferred_elution': num_elution,
    'rows_labeled_unknown': num_unknown
})

summary_df = pd.DataFrame(results_summary)

# Efficient log generation using vectorized string formatting
with open(OUTPUT_LOG, 'w') as f:
    f.write("# Preprocessing Log\n\n")
    f.write("Summary per run_no:\n")

```

```

f.write(f"{'run_no':<10}|{'Total_Rows':<12}|{'Excluded_(<1ml)':<16}|{'Inferred_Elution':<16}|{'Labeled_Unknown':<15}\n")
f.write("-" * 80 + "\n")

# Vectorized row formatting to avoid Python loop overhead
lines = summary_df.apply(
    lambda row: f"{'int(row['run_no']):<10}|{'int(row['total_rows_processed']):<12}|{'int(row['rows_excluded_post_baseline']):<16}|{'int(row['rows_inferred_elution']):<16}|{'int(row['rows_labeled_unknown']):<15}]",
    axis=1
)
f.write("\n".join(lines) + "\n")

print(f"Preprocessing complete. Log saved to {OUTPUT_LOG}")

if __name__ == "__main__":
    process_chromatography_data()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from scipy.signal import find_peaks
import matplotlib.pyplot as plt
import os

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
input_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(input_path)

# Step 1: Filter data to include only rows where chromatography_stage is 'Elution'
df_elution = df[df['chromatography_stage'] == 'Elution'].copy()

# Prepare output structures
peak_metrics_list = []

# Process each run
for run_no in df_elution['run_no'].unique():
    run_data = df_elution[df_elution['run_no'] == run_no].sort_values('volume_ml')
    .reset_index(drop=True)

    if run_data.empty:
        continue

    # Feedback: Ensure #Fraction_unique_ID# is filled forward correctly by
    # checking for leading NaNs
    if run_data['Fraction_unique_ID'].isna().all():
        run_data['Fraction_unique_ID'] = 'Unknown'
    else:
        run_data['Fraction_unique_ID'] = run_data['Fraction_unique_ID'].ffill()

# Step 2: Define pre-elution region (1.0 to 5.0 ml) to calculate baseline
# noise SD
pre_elution_mask = (run_data['volume_ml'] >= 1.0) & (run_data['volume_ml'] <=

```

```

5.0)
pre_elution_data = run_data[pre_elution_mask]

# Preliminary peak detection in pre-elution region to mask potential peaks
if len(pre_elution_data) > 0:
    pre_signal = pre_elution_data['UV_1_280_mAU'].values
    if len(pre_signal) > 2:
        pre_std = np.std(pre_signal)
        low_threshold = max(1.5 * pre_std, 0.1)
        pre_peaks, _ = find_peaks(pre_signal, height=low_threshold, distance
                                =1)

        pre_peak_mask = np.zeros(len(pre_elution_data), dtype=bool)
        for p_idx in pre_peaks:
            start = max(0, p_idx - 2)
            end = min(len(pre_elution_data), p_idx + 3)
            pre_peak_mask[start:end] = True

        clean_pre_data = pre_elution_data[~pre_peak_mask]
    else:
        clean_pre_data = pre_elution_data

    if len(clean_pre_data) > 0:
        baseline_noise_sd = clean_pre_data['UV_1_280_mAU'].std()
    else:
        baseline_noise_sd = run_data['UV_1_280_mAU'].std()
else:
    baseline_noise_sd = run_data['UV_1_280_mAU'].std()

# Step 3: Detect peaks
# Feedback: Handle potential #NaN# values in the #signal# array before
# performing comparisons
signal = np.nan_to_num(run_data['UV_1_280_mAU'].values, nan=0.0)
volumes = run_data['volume_ml'].values

height_threshold = 3 * baseline_noise_sd

if len(volumes) > 1:
    vol_diffs = np.diff(volumes)
    median_vol_step = np.median(vol_diffs)
    # Feedback: Add a robust check for #median_vol_step# being zero or very
    # close to zero
    width_samples = 0.5 / median_vol_step if median_vol_step > 1e-9 else 1
else:
    width_samples = 1

peaks, properties = find_peaks(
    signal,
    height=height_threshold,
    distance=max(1, int(width_samples))
)

valid_peaks = []
valid_props = {}

if len(peaks) > 0:
    if 'widths' in properties:
        valid_indices = []
        for i, p_idx in enumerate(peaks):

```

```

        width_samples_val = properties['widths'][i]
        width_ml = width_samples_val * median_vol_step
        if width_ml > 0.5:
            valid_indices.append(i)

    if valid_indices:
        valid_peaks = peaks[valid_indices]
        valid_props = {k: v[valid_indices] for k, v in properties.items()}
    else:
        valid_peaks = []
        valid_props = {}
else:
    valid_peaks = peaks
    valid_props = properties

# Step 4 & 5: Integrate AUC and Assign to Fractions
fraction_ranges = run_data.groupby('Fraction_unique_ID')['volume_ml'].agg(['min', 'max']).reset_index()
fraction_ranges.columns = ['Fraction_unique_ID', 'vol_min', 'vol_max']

current_peak_id = 1

# Feedback: Store calculated fallback peak boundaries to avoid redundant
# recalculations
stored_boundaries = {}

for i, peak_idx in enumerate(valid_peaks):
    # Determine start and end volume of the peak
    if 'left_bases' in valid_props and 'right_bases' in valid_props:
        left_base_idx = valid_props['left_bases'][i]
        right_base_idx = valid_props['right_bases'][i]
    else:
        # Feedback: Optimize Fallback Boundary Search (Vectorized)
        left_base_idx = peak_idx
        right_base_idx = peak_idx

        # Expand left: Vectorized search
        left_slice = signal[:peak_idx+1]
        threshold_left = signal[peak_idx] - height_threshold * 0.5
        valid_left_indices = np.where(left_slice < threshold_left)[0]
        # Feedback: Add checks for empty #valid_left_indices# and #
        # valid_right_indices#
        if len(valid_left_indices) > 0:
            left_base_idx = valid_left_indices[-1]
        else:
            left_base_idx = peak_idx

        # Expand right: Vectorized search
        right_slice = signal[peak_idx:]
        threshold_right = signal[peak_idx] - height_threshold * 0.5
        valid_right_indices = np.where(right_slice < threshold_right)[0]
        if len(valid_right_indices) > 0:
            right_base_idx = peak_idx + valid_right_indices[0]
        else:
            right_base_idx = peak_idx

    # Feedback: Ensure indices are within bounds (IndexError prevention)
    left_base_idx = max(0, min(left_base_idx, len(signal)-1))
    right_base_idx = max(0, min(right_base_idx, len(signal)-1))

```

```

# Store boundaries for plotting later
stored_boundaries[i] = (left_base_idx, right_base_idx)

start_vol = volumes[left_base_idx]
end_vol = volumes[right_base_idx]

# Extract peak segment
peak_segment = run_data.iloc[left_base_idx:right_base_idx+1]

# Calculate AUC
area_280 = np.trapz(peak_segment['UV_1_280_mAU'].values, peak_segment['
    volume_ml'].values)
area_260 = np.trapz(peak_segment['UV_2_260_mAU'].values, peak_segment['
    volume_ml'].values)

# Feedback: Check area_280 is not zero before calculating purity_ratio
if area_280 > 0:
    purity_ratio = area_260 / area_280
else:
    purity_ratio = 0.0

# Step 5: Assign to Fraction
# Feedback: Replace Nested Loop with Vectorized Intersection Logic
inter_min = np.maximum(start_vol, fraction_ranges['vol_min'])
inter_max = np.minimum(end_vol, fraction_ranges['vol_max'])

valid_inter_mask = inter_min < inter_max
valid_fractions = fraction_ranges[valid_inter_mask]

fraction_contributions = []

if not valid_fractions.empty:
    # Feedback: Eliminate slicing/copying in inner loop by iterating over
    # filtered view
    for idx, row in valid_fractions.iterrows():
        frac_id = row['Fraction_unique_ID']
        f_min = row['vol_min']
        f_max = row['vol_max']

        i_min = max(start_vol, f_min)
        i_max = min(end_vol, f_max)

        # Vectorized mask on peak_segment
        inter_mask = (peak_segment['volume_ml'] >= i_min) & (peak_segment[
            'volume_ml'] <= i_max)
        inter_data = peak_segment[inter_mask]

        if len(inter_data) > 0:
            contrib_280 = np.trapz(inter_data['UV_1_280_mAU'].values,
                inter_data['volume_ml'].values)
            fraction_contributions.append((frac_id, contrib_280))

# Feedback: Add a check for empty #valid_fractions# before iterating
associated_fraction = 'Unknown'
if fraction_contributions:
    fraction_contributions.sort(key=lambda x: x[1], reverse=True)
    max_contrib = fraction_contributions[0][1]
    top_frac = fraction_contributions[0][0]

```



```

        if len(fraction_contributions) > 1:
            second_contrib = fraction_contributions[1][1]
            diff = abs(max_contrib - second_contrib)
            if diff < (0.01 * max_contrib):
                associated_fraction = 'Continuous'
            else:
                associated_fraction = top_frac
        else:
            associated_fraction = top_frac

    peak_metrics_list.append({
        'run_no': run_no,
        'Peak_ID': current_peak_id,
        'Start_Volume': start_vol,
        'End_Volume': end_vol,
        'Area_280': area_280,
        'Area_260': area_260,
        'Purity_Ratio': purity_ratio,
        'Associated_Fraction_ID': associated_fraction
    })
    current_peak_id += 1

# Step 6: Visualisation
plt.figure(figsize=(10, 6))
plt.plot(run_data['volume_ml'], run_data['UV_1_280_mAU'], label='Raw_Signal',
         color='blue', alpha=0.7)
# Feedback: Add visualization for #UV_2_260_mAU# in the plot
plt.plot(run_data['volume_ml'], run_data['UV_2_260_mAU'], label='UV_2_260_mAU',
         color='green', alpha=0.5, linestyle='--')

if len(pre_elution_data) > 0:
    baseline_mean = pre_elution_data['UV_1_280_mAU'].mean()
    plt.axhline(y=baseline_mean, color='gray', linestyle='--', label='Baseline',
                ')

# Feedback: Reuse stored boundaries instead of recalculating
for i, peak_idx in enumerate(valid_peaks):
    if i in stored_boundaries:
        left_base_idx, right_base_idx = stored_boundaries[i]
    else:
        left_base_idx = peak_idx
        right_base_idx = peak_idx

    # Feedback: Add a check in the plotting loop to ensure #left_base_idx !=
    # right_base_idx#
    if left_base_idx != right_base_idx:
        plt.axvline(x=volumes[left_base_idx], color='red', linestyle=':',
                    label='Peak_Boundary' if i==0 else "")
        plt.axvline(x=volumes[right_base_idx], color='red', linestyle=':',
                    label="" if i==0 else "")
    else:
        pass

plt.xlabel('Volume_(ml)')
plt.ylabel('UV_(mAU)')
plt.title(f'Peak_Detection_and_Integration_Boundaries_-_Run_{run_no}')
plt.legend()
plt.grid(True, alpha=0.3)

```

```

        output_plot_path = f'/workspace/peak_detection_run_{run_no}.png'
        plt.savefig(output_plot_path)
        plt.close()

# Generate Summary Table for peak_metrics.md
df_metrics = pd.DataFrame(peak_metrics_list)

if not df_metrics.empty:
    df_metrics_sorted = df_metrics.sort_values(by=['run_no', 'Area_280'],
        ascending=[True, False])
    top_peaks = df_metrics_sorted.groupby('run_no').head(5)

    md_lines = ["|_Peak_ID_|_Start_Volume_|_End_Volume_|_Area_280_|_Purity_Ratio_|
        "]
    md_lines.append("
        |-----|-----|-----|-----|-----|")

    for _, row in top_peaks.iterrows():
        md_lines.append(f"|_{row['Peak_ID']}_|_{row['Start_Volume']:.2f}|_{row['
            End_Volume']:.2f}|_{row['Area_280']:.2f}|_{row['Purity_Ratio']:.2f}|_")

    if len(md_lines) > 20:
        md_lines = md_lines[:20]

    output_md_path = '/workspace/peak_metrics.md'
    # Feedback: Fix the file appending logic by checking if the file is empty
    # before seeking to the last character
    file_exists = os.path.isfile(output_md_path)
    with open(output_md_path, 'a') as f:
        if not file_exists:
            f.write('\n'.join(md_lines) + '\n')
        else:
            # Check if file is empty
            f.seek(0, 2) # Go to end
            file_size = f.tell()
            if file_size > 0:
                f.seek(file_size - 1)
                last_char = f.read(1)
                if last_char != '\n':
                    f.write('\n')
                f.write('\n'.join(md_lines) + '\n')

if not df_metrics.empty:
    df_metrics.to_csv('/workspace/peak_detection_results.csv', index=False)

```

Listing 2: Code.2

```

import pandas as pd
import os

input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/debug_columns.txt'

try:
    df = pd.read_csv(input_file)

    with open(output_file, 'w') as f:

```

```

        f.write("==_Column_Names_==\n")
        f.write(str(df.columns.tolist()) + "\n\n")
        f.write("==_First_5_Rows_==\n")
        f.write(df.head().to_string() + "\n\n")
        f.write("==_Data_Types_==\n")
        f.write(df.dtypes.to_string() + "\n")

    print(f"Debug_information_saved_to_{output_file}")
except Exception as e:
    with open(output_file, 'w') as f:
        f.write(f"Error_reading_file:{str(e)}")
    print(f"Error:{str(e)}")

```

Listing 3: Code_3